

פרויקט מערכות הפעלה – סימולציית תנועה בגרף

דרישות טכניות לכל אבני הדרך

כל repository חייב לכלול:

Makefile עם targets אחידים:

- make milestone4 בונה את milestone 4
- make milestone5 בונה את milestone 5 וכן הלאה
- make clean מנקה את כל קבצי הקימפול

פקודות הרצה:

```
./sim <file_name> # milestones 2–6
```

בשורש ה-repository קובץ README הכולל:

- פקודת קימפול והרצה לכל milestone
- תיאור קצר של המימוש בכל שלב

חשוב: הגשה שה Makefile שלה לא עובד או שחסר בה README לא תיבדק.

אבן דרך 4 : תהליכים מרובים ותהליך אב

תזמון : שבוע 4 🕒

המטרה

לעבור ממודל של נוסע יחיד למספר נוסעים שנעים בו-זמנית. תהליך האב מנהל את ה-GUI ומחשב מסלולים עבור תהליכי הבנים. למעשה, בשלב זה האבא עושה את כל העבודה (מחשב את מסלול הבנים ומציג אותם ב-GUI), הבנים רק נדרשים "לחיות" בזמן בו הם אמורים לנסוע על הגרף.

דרישות

- קובץ קלט מורחב: מספר בנים, לכל אחד node_dest node_source (ראו דוגמה בהמשך)
- תהליך האב:
 - קורא את הקובץ ומחשב את מסלול דייקסטרה לכל בן
 - יוצר את תהליכי הבנים באמצעות fork()
 - מנהל את לולאת raylib ומציג את כל הנוסעים על המסך
 - בשלב זה יש לייצג כל נוסע בצבע שונה ב-GUI
 - ממתין לסיום כל הבנים
- תהליכי הבנים:
 - האב מחשב לכל בן את המסלול לפני יצירתו
 - כל בן ידפיס לטרמינל מיד לאחר יצירתו:
- [PID] started
- לאחר מכן הבנים ישנים ולא עושים דבר
- הנסיעות של כל הנוסעים מתרחשות בו-זמנית על המסך
- בסיום המסלול של כל נוסע, האבא ישלח סיגנל לתהליך הבן ויסיים אותו.

פורמט קובץ קלט מורחב

```
# graph definition (same as before)
6 8
...
# travelers
3
0 5
1 4
2 3
```

קריטריונים לבדיקה עצמית

- שימוש בפקודה fork() ליצירת תהליכי הבנים
- כל נוסע מוצג בצבע שונה ב-GUI
- הנסיעות מתרחשות במקביל (לא אחת אחרי השניה)
- האב ממתין לכל הבנים לפני סיום
- יש לתייג (tag) ב-github: milestone4

אבן דרך 5 : תקשורת בין תהליכים (IPC)

תזמון : שבוע 5

המטרה

לעבור למודל שבו הבנים הם אוטונומיים: הם מחשבים את המסלול שלהם בעצמם ומדווחים לאב על מצבם. האב מציג את הכל ב-GUI על פי הדיווחים שמקבל מתהליכי הבנים ומדפיס log למסך המתאר את מהלך הריצה (פירוט בהמשך)

שינוי מבני מול שלב 4

בשלב 4, האב חישב את המסלול ומסר לבנים. בשלב 5, כל בן מחשב את המסלול שלו בעצמו (הגרף עדיין מגיע מהקובץ) ושולח את מיקומו במסלול לאבא באמצעות pipe או shared memory.

דרישות

- כל בן מחשב את מסלול דייקסטרה עבור עצמו ו"נוסע" במסול
- בכל פעם שכן מגיע לצומת חדש הוא שולח הודעה לאב בפורמט שתבחרו
- האב:
 - מקבל הודעות מהבנים
 - מעדכן את מיקום הבן בgui
 - מדפיס לוג לטרמינל: Y | next node: X | arrived at node [PID]
- כאשר X,Y הם מספרי הצמתים. ההדפסה תופיע עבור כל צומת בנסיעה (ראו דוגמה לפורמט הפלט בהמשך)
- ההדפסות למסך מגיעות מתהליך האב בלבד (הבנים כבר לא מדפיסים [PID] started)
- יש לציין בבירור בREADME איזה כלי IPC נבחר ולמה

דוגמה לקלט ופלט

עבור קובץ הקלט הבא:

```
# graph definition
5 7
0 1 4
0 2 2
1 3 5
2 1 1
2 3 8
3 4 2
1 4 6
# travelers
2
0 4
2 3
```

כך יראו ההדפסות למסך:

```
[PID=1021] arrived at node 0 | next node: 2
[PID=1022] arrived at node 2 | next node: 1
[PID=1021] arrived at node 2 | next node: 1
[PID=1022] arrived at node 1 | next node: 3
[PID=1021] arrived at node 1 | next node: 4
[PID=1022] arrived at node 3 | DESTINATION
[PID=1021] arrived at node 4 | DESTINATION
[PID=1022] finished
[PID=1021] finished
```

קריטריונים לבדיקה עצמית

- נתוני המסלול לא עוברים מהאב לבן
- כל הודעה שנשלחת ע"י בן מתקבלת ע"י האב ופלט מתאים מודפס למסך
- ה-log בטרמינל מציג את כל האירועים בסדר נכון (כמובן שהפלט יכול להשתנות כאשר הבנים מבצעים הדפסות במקביל)
- יש לתייג (tag) ב-github: milestone5

